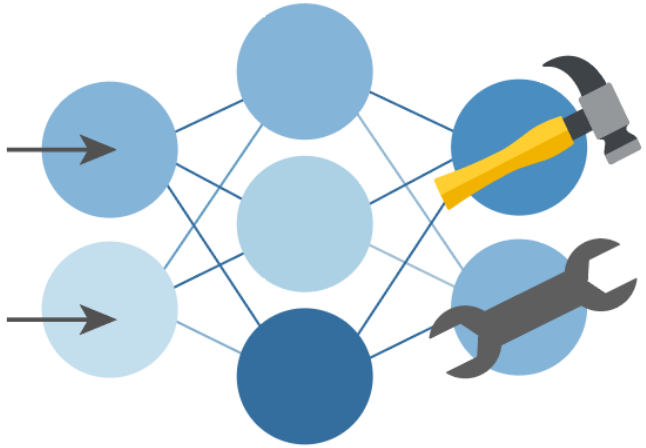
Python + Agents



🛠️ Feb 24: Building your first agent in Python
🧠 Feb 25: Adding context and memory to agents
📊 Feb 26: Monitoring and evaluating agents
🔀 Mar 3: Building your first AI-driven workflows
🕸️ Mar 4: Orchestrating advanced multi-agent workflows
🙋🏽‍♀️ Mar 5: Adding a human-in-the-loop to workflows

🔗 **Register at aka.ms/PythonAgents/series**

Microsoft

Python + Agents

Building your first AI-driven workflows

aka.ms/pythonagents/slides/workflows

Pamela Fox

Python Cloud Advocate

www.pamelafox.org

Today we'll cover...

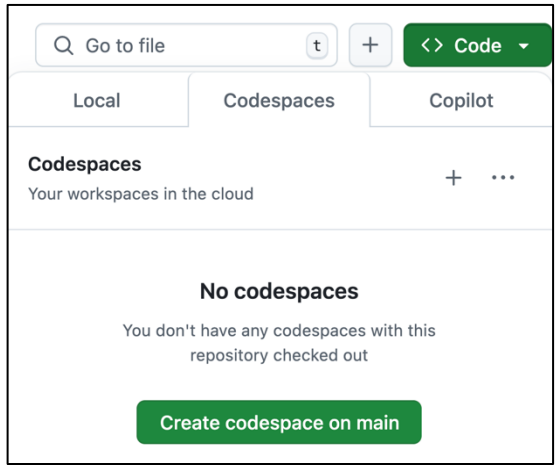
- Workflows: executors, edges, messages, and outputs
- Building a basic sequential workflow
- Using agents as workflow executors
- Visualizing workflow runs with DevUI
- Branching with conditional edges and switch-case routing
- Structured outputs for more reliable workflow routing
- Workflow state management
- Full-stack web application with an agentic workflow

Want to follow along?

1. Open this GitHub repository:

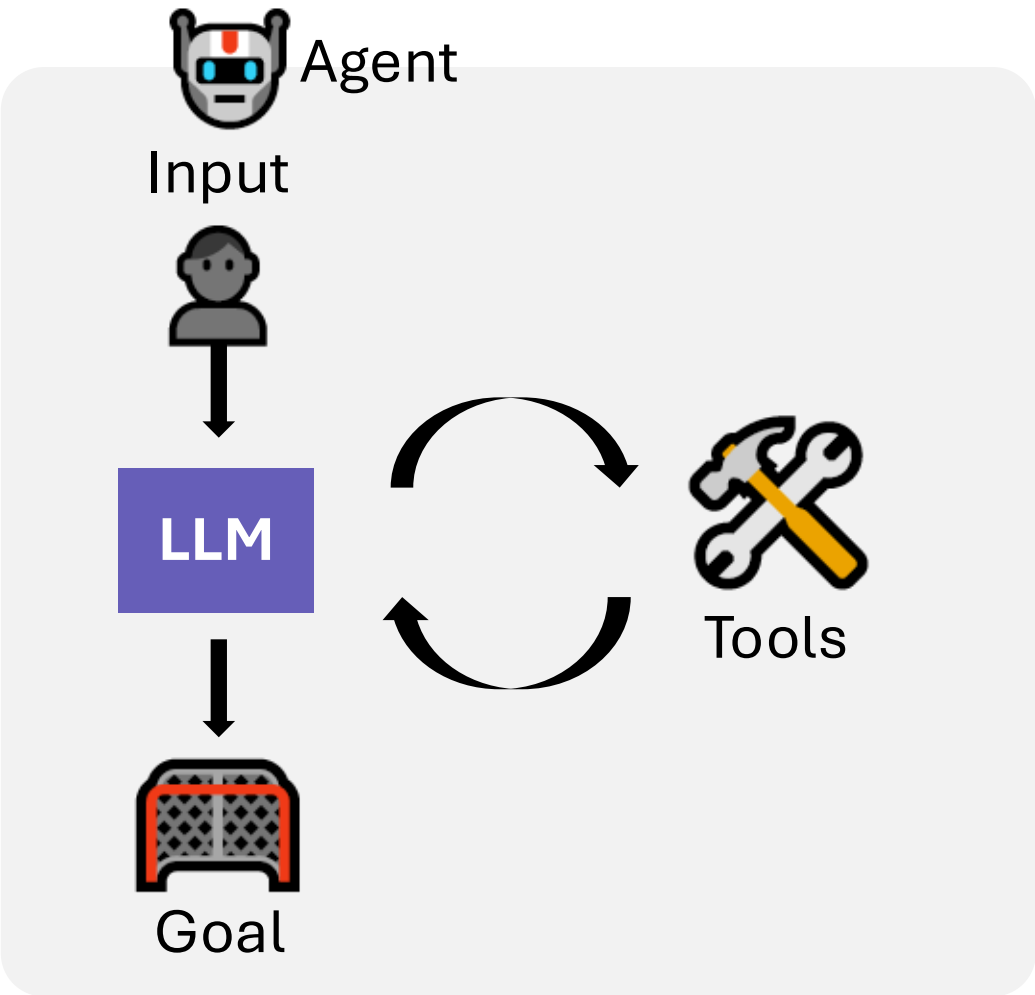
aka.ms/python-agentframework-demos

2. Use "Code" button to create a GitHub Codespace:



3. Wait a few minutes for Codespace to start up ⏱️
4. Run `git pull` to make sure you have the latest code

Recap: What's an agent?

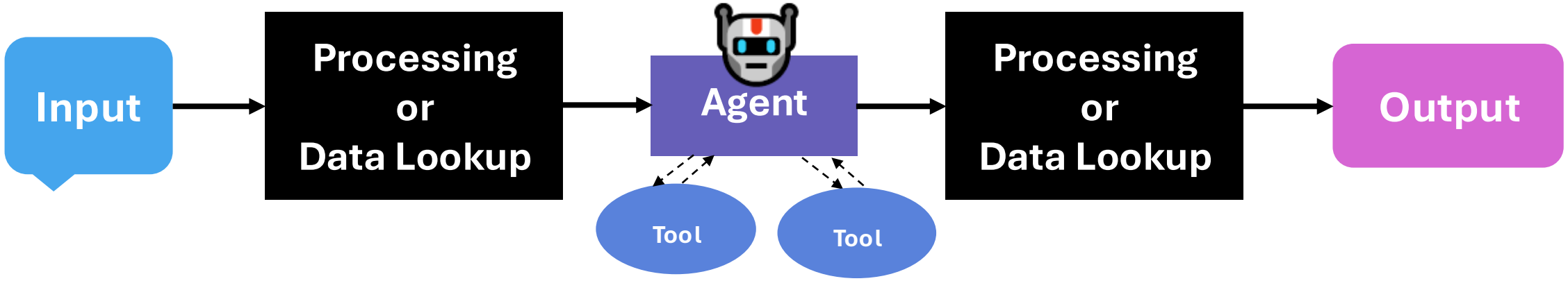


An **AI agent** uses an **LLM** to run **tools** in a loop to achieve a **goal**.

Workflows

What's a workflow?

An agentic workflow is any flow that involves an agent at some point, typically to handle decision making or answer synthesis.



Not all calls in our systems *should* be agentic!
Every call to an LLM increases the non-determinism of a system and possibility of failure or safety risk. 😬

agent-framework

Anatomy of a workflow

A workflow is a graph with `Executor` nodes and edges between them:

Each executor subclass defines handler methods that **receive messages** and may **send messages to next executor** and/or **yield outputs:**

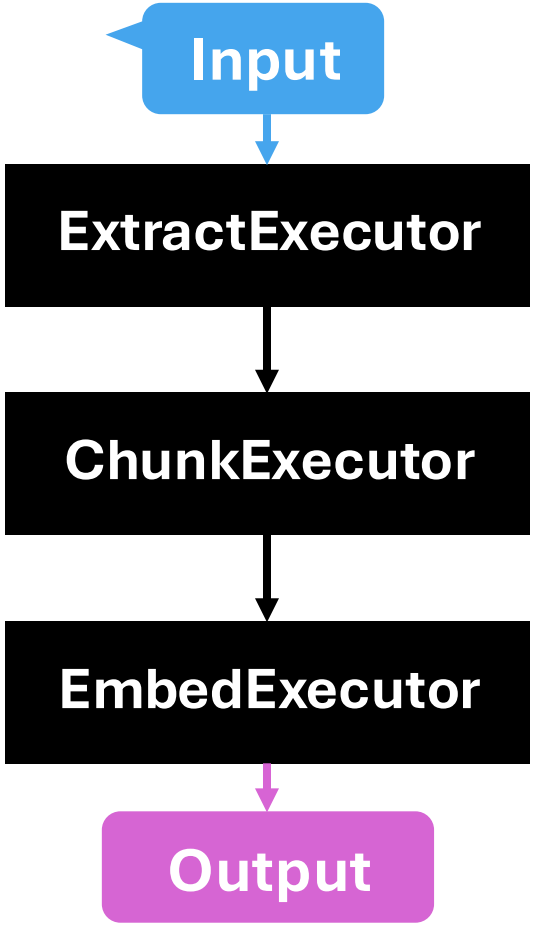
```
class SampleExecutor(Executor):
  @handler
  async def sample_handler(self, text: str, ctx: WorkflowContext[str]):
    await ctx.send_message(text.upper())

class OutputExecutor(Executor):
  @handler
  async def sample_handler(self, text: str, ctx: WorkflowContext[Never, str]):
    await ctx.yield_output(text)
```

agent-framework

Building a workflow

Use `WorkflowBuilder()` to connect instances of `Executor()` subclasses:

```
from agent_framework import WorkflowBuilder

extract = ExtractExecutor(id="extract")
chunk = ChunkExecutor(id="chunk")
embed = EmbedExecutor(id="embed")

workflow = WorkflowBuilder(start_executor=extract)
  .add_edge(extract, chunk)
  .add_edge(chunk, embed).build()

events = await workflow.run(pdf_path)
outputs = events.get_outputs()
```



Full example: [workflow_rag_ingest.py]

agent-framework

Visualize workflows with DevUI

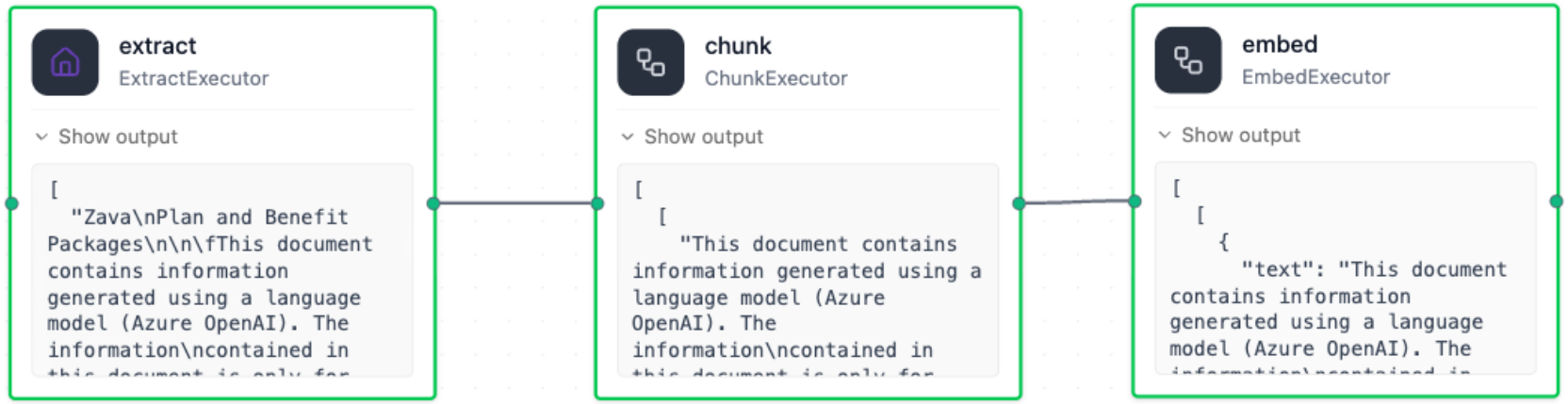
Install the devui package:

```
agent-framework-devui
```

Point devui to the workflow:

```
from agent_framework.devui import serve
serve(entities=[workflow], auto_open=True)
```

Run the workflow and see the progress through each node:



agent-framework

Workflows with Agent executors

Every `Agent` instance can also be used as an `Executor` in a workflow:

```
writer = Agent(client=client,
  name="Writer",
  instructions="You are a concise content writer.")

reviewer = Agent(client=client,
  name="Reviewer",
  instructions="Review content based on clarity, accuracy, and structure.")

workflow = WorkflowBuilder(start_executor=writer)
             .add_edge(writer, reviewer)
             .build()
```

Full example: [workflow_agents.py]

agent-framework

Streaming events from workflows

Emit events from workflows by specifying `stream=True`:

```
async for event in workflow.run(prompt, stream=True):

  if event.type == "started":
    print(f"[started] for workflow {workflow.name}")

  elif event.type == "executor_invoked":
    print(f"[executor_invoked] from executor {event.executor_id}")

  elif event.type == "output" and isinstance(event.data, AgentResponseUpdate):
    print(event.data.text, end="", flush=True)

  elif event.type == "executor_completed":
    print(f"[executor_completed] from executor {event.executor_id}")

  elif event.type == "executor_failed":
    print(f"[executor_failed] from executor {event.executor_id}:  {event.data}")

  elif event.type == "error":
    print(f"[error] {event.data}")
```

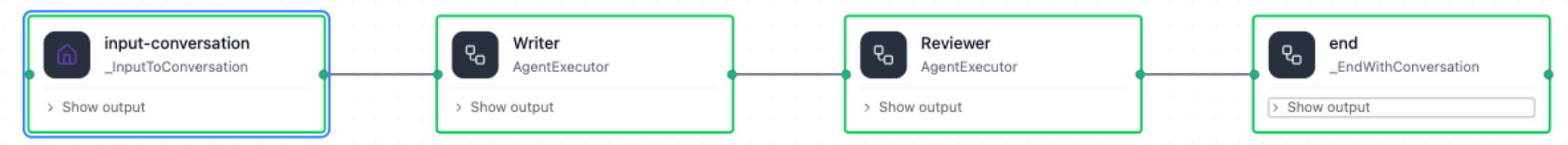
Full example: [workflow_agents_streaming.py]

agent-framework

Built-in workflow builder: SequentialBuilder

If you're building a linear conversation pipeline, you can use `SequentialBuilder`:

```
from agent_framework.orchestrations import SequentialBuilder

workflow = SequentialBuilder(participants=[writer, reviewer])
              .build()
```



The workflow assumes that participants accept and emit conversation-shaped data, so executors must be `Agent`s or custom `Executor`s that process conversations.

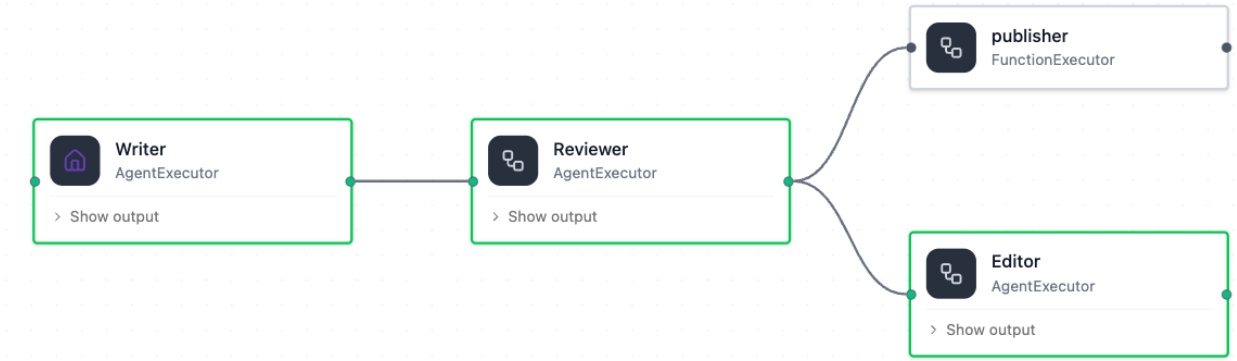
The extra start/end nodes normalize input and output to `list[Message]`.

Full example: [workflow_agents_sequential.py]

Branching workflows

agent-framework

Workflow with conditional branching



Each edge in a graph can have a condition (function that returns True/False).

```
def needs_revision(message) -> bool:
  return message.agent_response.text.upper().startswith("REVISION NEEDED")
def is_approved(message) -> bool:
  return message.agent_response.text.upper().startswith("APPROVED")

workflow = (WorkflowBuilder(start_executor=writer)
  .add_edge(writer, reviewer)
  .add_edge(reviewer, publisher, condition=is_approved)
  .add_edge(reviewer, editor, condition=needs_revision)
  .build())
```

Full example: [workflow_conditional.py]

Text output is fragile for workflow control

System prompt for the `Reviewer` agent:

```
Your response MUST begin with exactly one of these two tokens:
APPROVED — if the draft is clear, accurate, and well-structured.
REVISION NEEDED — if it requires improvement.
```

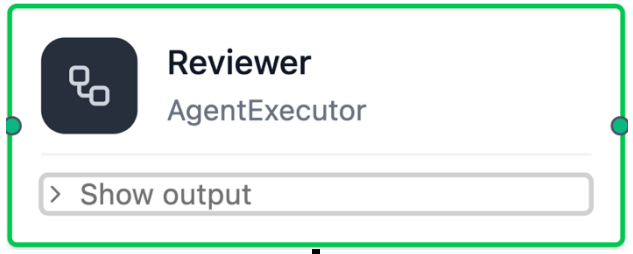
Condition functions that route based on `Reviewer` response:

```python
def needs_revision(message) -> bool:
  return message.agent_response.text.upper().startswith("REVISION NEEDED")
def is_approved(message) -> bool:
  return message.agent_response.text.upper().startswith("APPROVED")
```

What can go wrong?		Result:
Leading whitespace/newline/prefix	`Review result: APPROVED`	⁉️ No condition matched
Localization	`APROBADO`	⁉️ No condition matched
False positive approval	`APPROVED, but major issues remain..`	😱 Routes to wrong path
Mixed output	`APPROVED....REVISION REQUIRED...`	🤔 Unclear intent

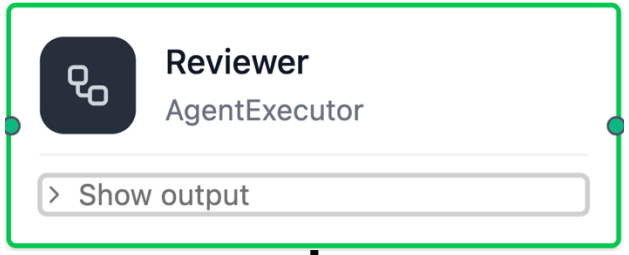
Unstructured text vs. structured output

The default output for an LLM is free text:




```
REVISION NEEDED

Problems to fix
- Remove em dashes. Replace e
ach "—" with a comma, semicol
on, or parentheses.".
```

If an LLM supports *structured outputs*, you can force output to conform to schema:




```
{
 "decision": "REVISION NEEDED",
 "feedback": "Problems to fix: Rem
ove em dashes. Replace each "—" wi
th a comma, semicolon, or parenthe
ses."
}
```

agent-framework

Using structured outputs for workflow decisions

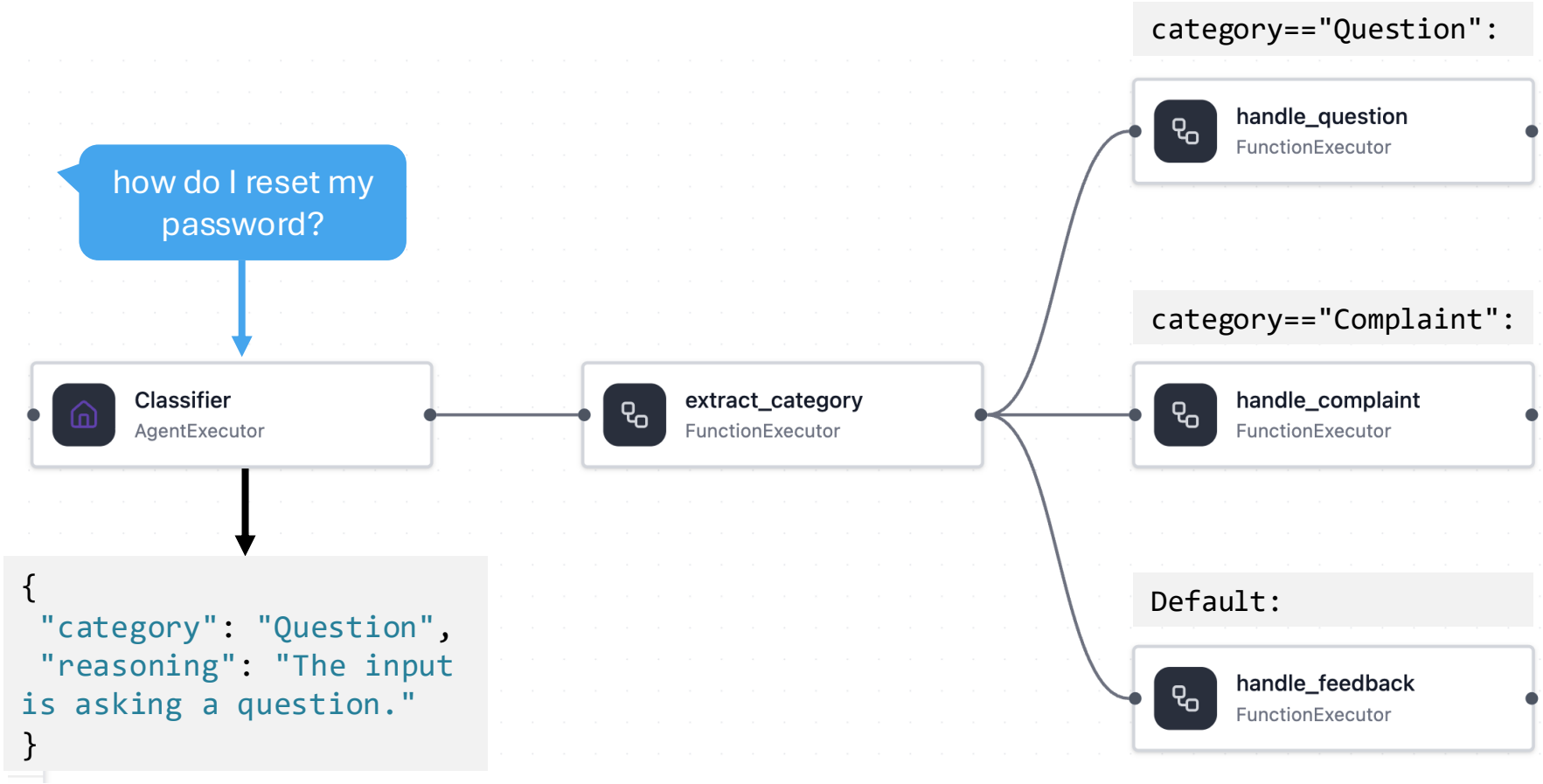
Specify `response_format` for `Agent` and convert response to that model:

```
class ReviewDecision(BaseModel):
  decision: Literal["APPROVED", "REVISION_NEEDED"]
  feedback: str
  final_post: str | None = None

reviewer = Agent(client=client,
  name="Reviewer",
  instructions=("You are a strict content reviewer."
  "If draft is ready, set decision=APPROVED and include final post in final_post. "
  "If it needs changes, set decision=REVISION_NEEDED and provide feedback."),
  response_format=ReviewDecision)

def is_approved(message: Any) -> bool:
  result = ReviewDecision.model_validate_json(message.agent_response.text)
  return result.decision == "APPROVED"
```

Full example: [workflow_conditional_structured.py]

agent-framework

Category-based routing with structured outputs

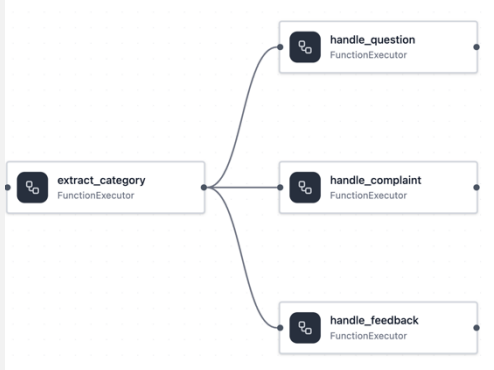


agent-framework

Using switch-case edges in workflows

Add a switch-case edge group with an executor that categorizes output:

```
classifier = Agent(..., response_format=ClassifyResult)

@executor(id="extract_category")
async def extract_category(response, ctx):
  result = ClassifyResult.model_validate_json(response.agent_response.text)
  await ctx.send_message(result)

workflow = (WorkflowBuilder(start_executor=classifier)
  .add_edge(classifier, extract_category)
  .add_switch_case_edge_group(extract_category,
   [Case(condition=is_question, target=handle_question),
    Case(condition=is_complaint, target=handle_complaint),
    Default(target=handle_feedback)])
  .build())
```



Full example: [workflow_switch_case.py]

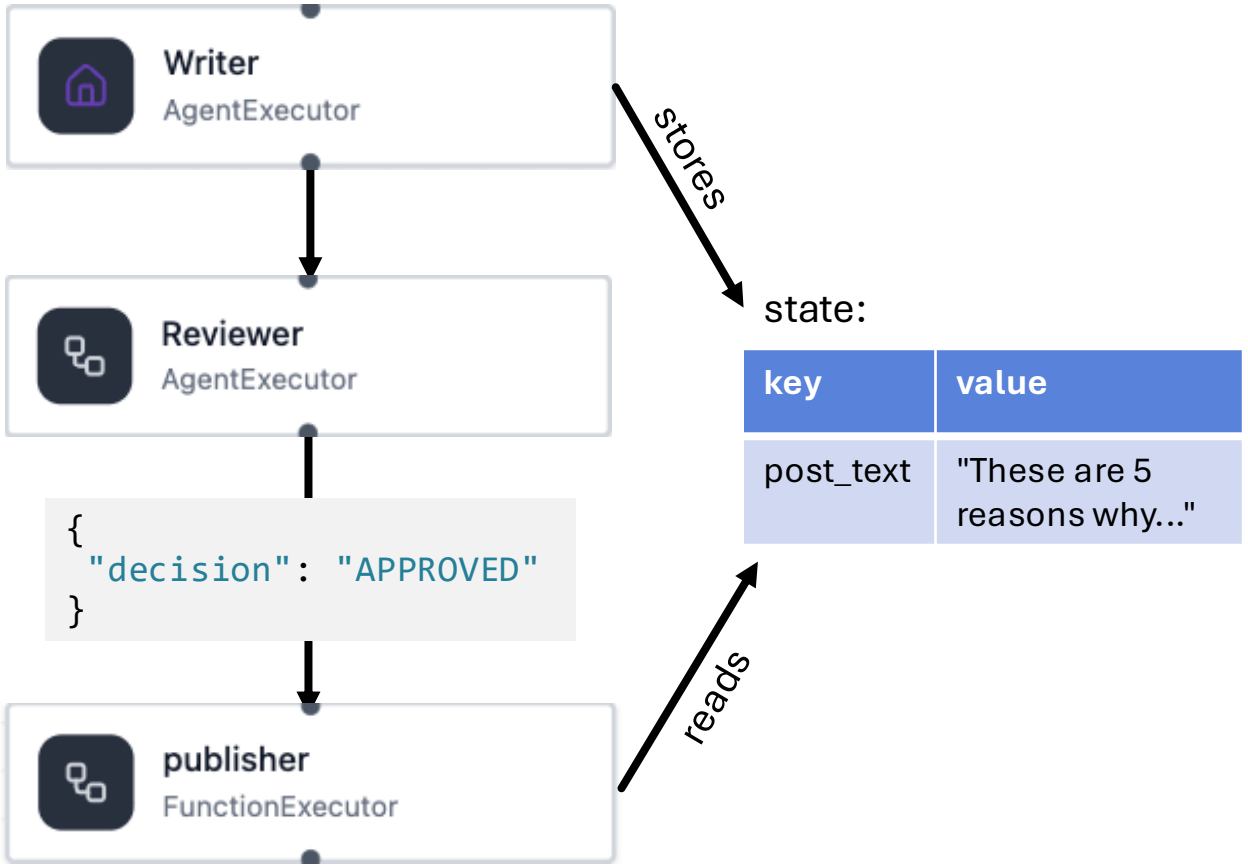
State management

When do we need workflow state?

Without state:

Reviewer must carry content that it didn't produce, just to pass it downstream.

With state:



key	value
post_text	"These are 5 reasons why..."

Reviewer only decides. Draft is stored once and read by any step that needs it.

agent-framework

Storing and accessing state in workflows

```
@executor(id="store_post_text")
async def store_post_text(response, ctx):
  ctx.set_state("post_text", response.agent_response.text.strip())
  await ctx.send_message(response)

@executor(id="publisher")
async def publisher(response, ctx):
  content = str(ctx.get_state("post_text", "")).strip()
  await ctx.yield_output(f"✅ Published:\n\n{content}")

workflow = (WorkflowBuilder(start_executor=writer, max_iterations=8)
    .add_edge(writer, store_post_text)
    .add_edge(store_post_text, reviewer)
    .add_edge(reviewer, publisher, condition=is_approved)
    .add_edge(reviewer, editor, condition=needs_revision)
    .add_edge(editor, store_post_text)
    .build())
```

store state

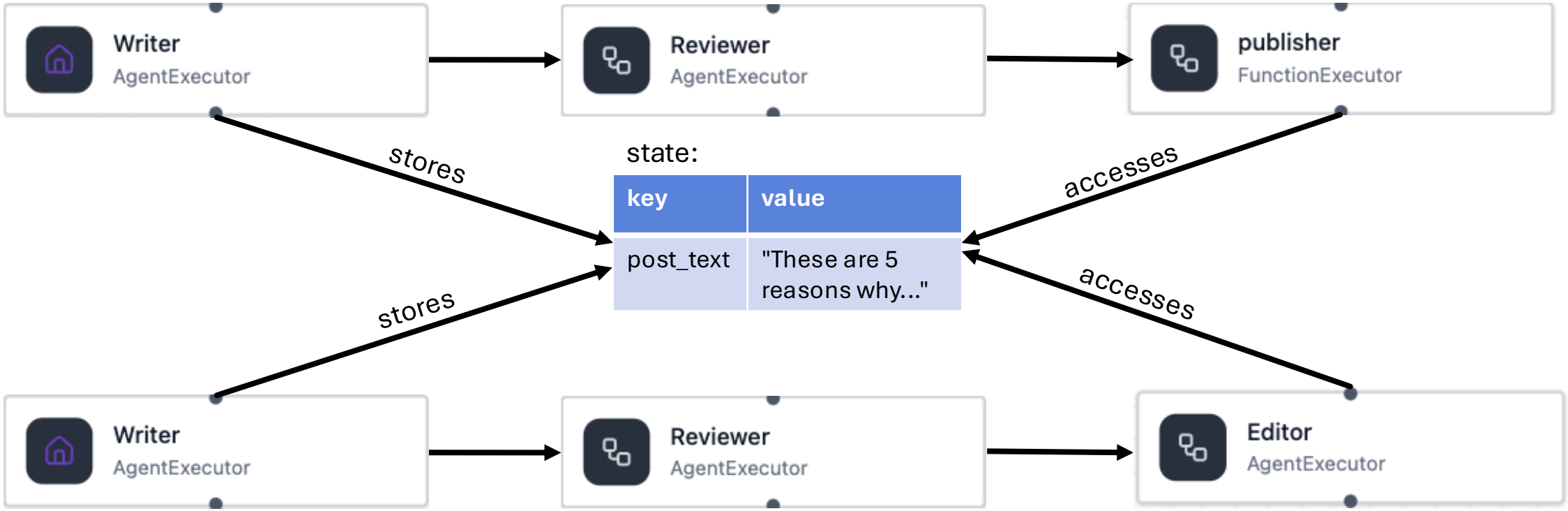
access state

Full example: [workflow_conditional_state.py]

Shared workflow state leaks between requests

Be careful about using a shared `Workflow` instance across multiple runs:

```
workflow = WorkflowBuilder(start_executor=writer, ...).build()
```



If two runs execute in parallel, they can run into a race condition!

⚠ Shared agents accumulate conversation history

Be careful about using a module-level Agent instance shared across workflow runs:

```
writer = Agent(...)
workflow = WorkflowBuilder(start_executor=writer, ...).build()
```

Run 1: "Write a post about AI jobs"

Writer
AgentExecutor

history:

```
user: "Write a post about AI jobs"
assistant: "AI agents are transforming..."
```

✅ Published: "AI agents are transforming..."

Run 2: "Write a post about remote work"

Writer
AgentExecutor

```
user: "Write a post about AI jobs"
assistant: "AI agents are transforming..."
user: "Write a post about remote work"
assistant: "Remote jobs are at risk due to AI.."
```

R1

R1

✅ Published: "Remote jobs are at risk due to AI..."

agent-framework

State isolation via factory functions

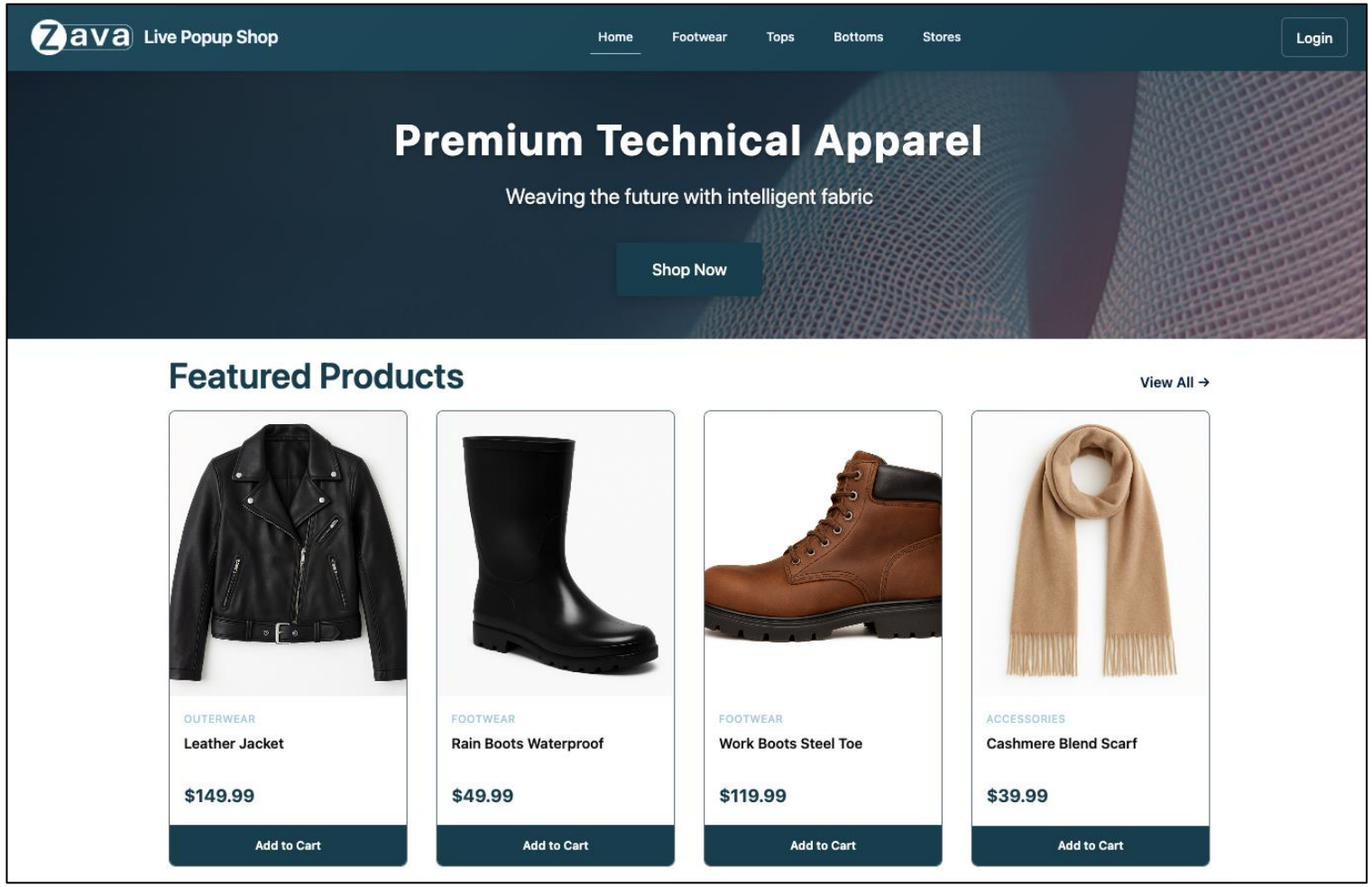
Use a factory function to create `Agent` instances and build the `Workflow`:

```
def create_workflow():
  writer = Agent(name="Writer", ...)
  reviewer = Agent(name="Reviewer", ...)
  editor = Agent(name="Editor", ...)

  return (WorkflowBuilder(start_executor=writer, max_iterations=8)
      .add_edge(writer, store_post_text)
      .add_edge(store_post_text, reviewer)
      .add_edge(reviewer, publisher, condition=is_approved)
      .add_edge(reviewer, editor, condition=needs_revision)
      .add_edge(editor, store_post_text)
      .build())

workflow = create_workflow(client)
events = await workflow.run(prompt)
```

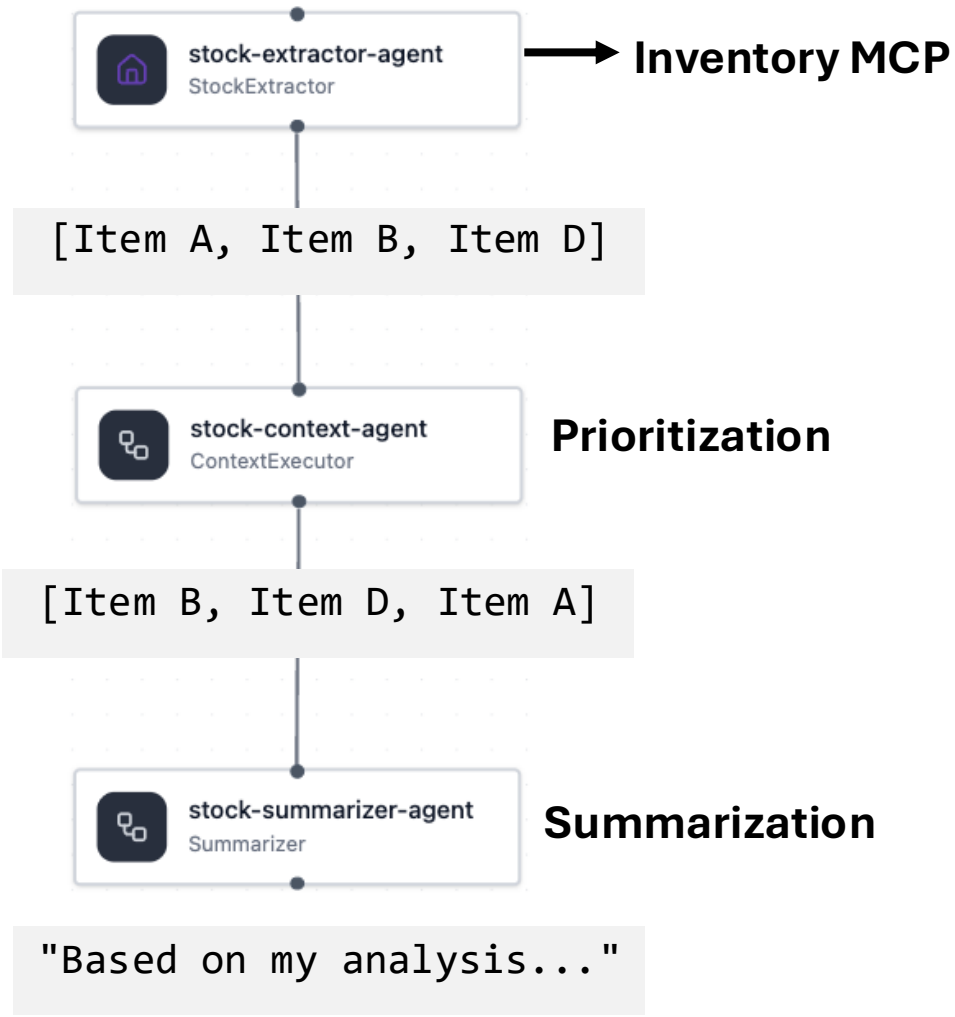
Full example: [workflow_conditional_state_isolated.py]

Using workflows in applications

Retail shop with agentic workflows



Sequential workflow: Restocking inventory



Restocking Recommendations 9 items need restocking. Select items to order.

AI Analysis Summary

Inventory Analysis Complete

Based on my analysis of the **Popup Downtown Redmond** store inventory, I've identified **9 products** that require immediate restocking attention:

Critical Low Stock Items

- **Thermal Winter Socks** and **Mesh Athletic Sneakers** both have only **2 units** remaining
- **Cozy Slipper Socks** has **4 units** in stock

Priority Recommendations

1. **Accessories**: 2 items need restocking (winter socks and slipper socks)
2. **Footwear**: 1 critical item (athletic sneakers)
3. **Apparel**: 3 items requiring attention (tees, shorts, jeans)
4. **Outerwear**: 3 items at moderate stock levels (9 units each)

Total estimated restock cost: Consider budgeting approximately **$500-600** for the recommended quantities.

SKU	PRODUCT NAME	CATEGORY	CURRENT STOCK	UNIT COST	ORDER QTY	TOTAL COST
ACC-SK-005	Thermal Winter Socks	Accessories	2	$14.99	10	$149.90
ACC-SK-008	Cozy Slipper Socks	Accessories	4	$15.99	10	$159.90

Next steps

Register:
https://aka.ms/PythonAgents/series

Watch past recordings:
aka.ms/pythonagents/resources

Join office hours after each session in Discord:
aka.ms/pythonai/oh

🛠️ Feb 24: Building your first agent in Python
🧠 Feb 25: Adding context and memory to agents
📊 Feb 26: Monitoring and evaluating agents
🔀 Mar 3: Building your first AI-driven workflows
🕸️ Mar 4: Orchestrating advanced multi-agent workflows
🙋🏽‍♀️ Mar 5: Adding a human-in-the-loop to workflows